Ron Hensley
ronkhensley@gmail.com

1. AAS in AI Software Engineering
2. PY101
3. Unlocking print() power-ups

Today

Welcome to your next lesson: Unlocking print() power-ups.

I'm preparing your lesson based on what you've already learned, what comes next, and the best way to help you grow.

Step by step, you're moving forward.

Ron, let's power up your print() skills today ?

Imagine you're printing a concert ticket and you want this on the screen:

Band: The PyTones

Date: 2026-06-01

Seat: A12

We'll learn how to:

- Put several things in one `print()` using commas
- See how each `print()` goes to a new line
- Use `+` and `*` inside `print()` to build fun outputs

To start simple: you already know this kind of thing:

```python
print("Hello, world!")
```

If you guess, what do you think this does:

```python
print("Band:", "The PyTones")
```

No need to run it yet—just guess in your head or tell me in a short sentence:

Does it show the words stuck together, spaced out, or something else?

I think they're spaced out

What do you think the right answer is in the question below? ?

What will this code print?

```
print("Band:", "The PyTones")
```

A.

Band:The PyTones

B.

Band: The PyTones

C.

Band: The PyTones (two spaces)

D.

"Band:", "The PyTones"

Yes! ?

print("Band:", "The PyTones" shows:

Text

```
Band: The PyTones
```

Step by step what happens:

- "Band:" is one value
- "The PyTones" is a second value
- The comma in print() tells Python: “print both, with a space between them”

So Band: and The PyTones come out with one space in between.

Next tiny step: each print() call automatically moves to a new line after it finishes.

So what do you expect this to look like on screen?

Python

```
print("Band:", "The PyTones")
print("Date:", "2026-06-01")
print("Seat:", "A12")
```

You don’t have to run it yet—just tell me: will these three lines appear

- under each other (three lines), or
- all on one long line?

I think it's three lines

Nice, that’s exactly right ✅

Each print() ends by moving to the next line automatically. So:

Python

```
print("Band:", "The PyTones")
print("Date:", "2026-06-01")
print("Seat:", "A12")
```

will show like this:

Text

```
Band: The PyTones
Date: 2026-06-01
Seat: A12
```

One print() = one line on the screen.

Next, I want to show you how print() behaves when we use + and * with text, but step by step.

First, a quick idea check in words (no code yet):

If you see this in Python:

Python

```
print("Hello, " + "Ron")
```

do you expect it to:

- glue the two pieces together into one phrase, or
- print them separately like with commas?

Just answer “glue together” or “separate like commas.”

Separate like commas

Great, this is the interesting difference ?

With commas:

```Python
print("Hello,", "Ron")
```

Python prints both pieces with a space between, like:

```Text
Hello, Ron
```

With `+`:

```Python
print("Hello, " + "Ron")
```

Python glues the text into one string first → `"Hello, Ron"` → then prints that single result. So the right mental model: `+` glues, commas separate with spaces.

Let’s try a tiny prediction before you code anything.

Think about this:

```Python
print("la" * 3)
```

Do you expect the output to look more like:

- la3
- lalala

Just pick one of those in your reply.

I think it's la3

Nice guess, and this is where Python might surprise you a bit ?

```Python
print("la" * 3)
```

actually prints:

```Text
lalala
```

Step by step:

- `"la"` is text
- `* 3` means “repeat this 3 times”
- So Python builds `"lalala"` first, then `print()` shows that

So for strings:

- `+` → glues text together

- * → repeats text several times

Let's get your hands on the keyboard for a moment ?

In the code editor, write and run this:

```
Python
print("Boom " * 3)
```

Hit Run (you don't need to Submit yet), look at the output, and tell me:

Did it match what you expected, or did it look different?

It matched my expectation

Love it — your mental model is lining up with what Python does ?

Quick recap in super simple form:

- **print**("Hello,", "Ron") → uses commas → prints both parts with a space between → two separate values
- **print**("Hello, " + "Ron") → uses + → glues the text into one value, then prints it
- **print**("Boom " * 3) → uses * with text → repeats the text → **Boom Boom Boom**

Now I want to check that all three power-ups are clear so far:

1. Commas in **print**()
2. New line after each **print**()
3. + and * with strings

Want to try a quick one-question check about + and * inside **print**()?

Yes, let's do it

What do you think the answer is for the question below? ?

What will this code print?

```
print("Ha" + "Ha" * 2)
```

Remember: + glues strings, * repeats strings.

A.

HaHaHaHa

B.

HaHa2

C.

HaHa

D.

"Ha" + "HaHa"

Nice work, Ron — you nailed that one ?

For:

```
Python
print("Ha" + "Ha" * 2)
```

Python does this in order:

1. `"Ha" * 2` → repeats → `"HaHa"`
2. `"Ha" + "HaHa"` → glues → `"HaHaHaHa"`
3. `print(...)` shows:
   HaHaHaHa

So you used both rules correctly: `*` repeats first, then `+` glues.

Let's do one more small challenge with commas + new lines + `*` all together ?

In the code editor, write and run this:

```
Python
print("Boom" * 3)
print("Boom", "Boom", "Boom")
```

Don't submit yet, just Run it.

Then tell me: do the two lines of output look the same or different?

> Facts check. You were saying that the interpretation works the same way as math multiplication before addition so the second and third term are multiplied first then the result is concatenated using the plus symbol with the first item, correct?

Exactly right — you're thinking about it in the same order as regular math ?

Step by step for:

```
Python
print("Ha" + "Ha" * 2)
```

1. Python sees `"Ha" * 2` → `*` has higher precedence (like in math)
2. It repeats the string: `"Ha" * 2` → `"HaHa"`
3. Then it does the `+`: `"Ha" + "HaHa"` → glues them → `"HaHaHaHa"`
4. `print()` shows that result

So yes: multiply first, then add, just like numbers.

Do you want to try another quick one like this, or go back to commas/new lines for a moment?

> I ran the boom boom boom code, the second example, and it was as I expected. The, version has spaces in between and the multiplication version was concatenated together as boomboomboom with no spaces

Perfect observation ?

You've got all three ideas locked in:

- `print("Boom" * 3)` → `"BoomBoomBoom"` (no spaces, because `*` just repeats the text exactly)
- `print("Boom", "Boom", "Boom")` → **Boom Boom Boom** (spaces added because commas in `print()` separate values with a space)
- Each `print(...)` still goes on its own line.

So you now understand:

1. Commas in `print()`
2. Automatic new line after each `print()`
3. `+` and `*` with strings inside `print()`

? Last mini challenge, just to tie it all together:

If you want the output to be exactly:

```
Boom Boom Boom
```

on one line, and then go to a new line after it, which is the best option?

A) `print("Boom" * 3)`

B) `print("Boom", "Boom", "Boom")`

Just answer A or B.

B

Exactly — B is the one ✅

```
print("Boom", "Boom", "Boom")
```

prints:

```
Boom Boom Boom
```

and then automatically moves to the next line when it's done.

The commas give you the spaces; `print()` gives you the newline.

You've now used:

- commas to print multiple values with spaces
- the fact that each `print()` goes to a new line
- `+` and `*` with text inside `print()`

You've basically mastered this lesson's goals ?

Want to wrap up this lesson and move on to the next one, or do you want one more challenge with `print()` before we finish?

Let's move on

Awesome work today, Ron ?

Here's what you're taking with you:

- You can use commas in `print()` to show multiple values with spaces between them, like a neat ticket line:
  `print("Band:", "The PyTones")` → Band: The PyTones
- You know that each `print()` call goes to a new line, so stacking prints stacks lines on the screen.
- You understand that with strings:

- + glues text pieces into one string
- * repeats text, like "Boom " * 3 → **Boom Boom Boom**

Maestro can make mistakes. Check important info.

You're thinking in the right "math order" way too (multiplication before addition), which will keep helping as we go deeper into Python.

When you're ready, go ahead and start the next lesson in the Learn tab — I'll keep building on this with you step by step ?